Streamlit

Projet Streamlit : Initiation 1
Jeu de données 1
Streamlit Pages: 1
Liste de commandes 1
Créer un environnement virtuel (nommé '.venv') 1
Activer l'environnement virtuel : 1
Installer Streamlit et Pandas : 1
Générer le fichier requirements.txt : 1
pour réinstaller les packet d'un requirements.txt 1
désactiver le venv. 1
Points à aborder 2
Projet : Exploration interactive des données de ventes 3
Objectif 3
Étapes détaillées 3
Installation et configuration (5 min) 3
Chargement et affichage des données (15 min) 3
Widgets interactifs (20 min) 3
Visualisation de données (adaptées aux colonnes disponibles) 4
Visualisation de données (20 min) 4
Mise en page (15 min) 4
Titre du projet : Visualisation interactive d'une fonction affine avec Streamlit 5
Objectif : 5
Fonctionnalités : 5
Instructions : 5

Aller plus loin! Au-delà de streamlit! 6
Alternative 6

Projet Streamlit : Initiation

Ce projet vous guidera pas à pas dans la création d'un dashboard interactif avec Streamlit. Vous commencerez par les bases et progresserez vers des fonctionnalités plus avancées, pour finalement obtenir une application complète de visualisation et d'analyse de données.

Jeu de données

Pour ce projet, vous pouvez utiliser un jeu de données de votre choix :

- Données sur le Titanic : https://www.kaggle.com/c/titanic/data
- Données sur les ventes d'une entreprise : https://www.kaggle.com/datasets/kyanyoga/sample-sales-data

Streamlit Pages:

- créer un dossier: “pages”
- dans “pages” créer un fichier 0_le_nom_de_ma_page.py

Liste de commandes

Créer un environnement virtuel (nommé '.venv')

```
python -m venv .venv
```

Activer l'environnement virtuel :

- Windows: `.venv\Scripts\activate`
- macOS/Linux: `source .venv/bin/activate`

Installer Streamlit et Pandas :

```
pip install streamlit pandas
```

Générer le fichier requirements.txt :

```
pip freeze > requirements.txt
```

pour réinstaller les packet d'un requirements.txt

```
pip install -r requirements.txt
```

désactiver le venv.

```
deactivate
```

Points à aborder

1. **Installation et configuration**
   - Installer Streamlit (`pip install streamlit`).
   - Créer un fichier `hello.py` et faites `import streamlit as st`
   - Lancer une application Streamlit simple (`streamlit run hello.py`).
2. **Affichage de texte**
   - Afficher un titre (`st.title()`).
   - Afficher des en-têtes (`st.header()`, `st.subheader()`).
   - Afficher du texte (`st.write()`).
   - Afficher du Markdown (`st.markdown()`).
3. **Chargement et affichage de données**
   - Charger un jeu de données avec Pandas (`pd.read_csv()`).
   - Afficher un DataFrame Pandas (`st.dataframe()`).
   - Afficher un tableau statique (`st.table()`).
4. **Widgets interactifs**
   - Créer un bouton (`st.button()`).
   - Créer une case à cocher (`st.checkbox()`).
   - Créer un champ de saisie de texte (`st.text_input()`).
   - Créer un sélecteur déroulant (`st.selectbox()`).
   - Créer un curseur (`st.slider()`).
5. **Mise en page**
   - Utiliser des colonnes (`st.columns()`).
   - Utiliser des conteneurs extensibles (`st.expander()`).
   - Utiliser des onglets (`st.tabs()`).
6. **Visualisation de données**
   - Créer des graphiques avec Matplotlib (`st.pyplot()`).
   - Créer des graphiques interactifs avec Plotly (`st.plotly_chart()`).
   - Créer des graphiques avec Seaborn (`st.pyplot()` après avoir créé le graphique avec Seaborn).
7. **États de session**
   - Utilisation de `st.session_state` pour stocker des informations entre les interactions de l'utilisateur.
8. **Gestion des fichiers**
   - Permettre aux utilisateurs de télécharger des fichiers (`st.file_uploader()`).
   - Enregistrer des fichiers générés par l'application (`st.download_button()`).
9. **Intégration avec d'autres bibliothèques**
   - Intégrer l'application avec d'autres bibliothèques Python (logging pour journaliser.).

Projet : Exploration interactive des données de ventes

Objectif

Créer une application Streamlit interactive qui permet aux utilisateurs d'explorer et de visualiser les données de ventes d'une entreprise.

Étapes détaillées

Installation et configuration (5 min)

- ➔ Ouvrir un terminal et créer un environnement virtuel (`.venv`).
- ➔ Activer l'environnement virtuel.
- ➔ Installer Streamlit et Pandas : `pip install streamlit pandas`.
- ➔ Créer un fichier `0_introduction.py` dans un dossier nommé `pages`.
- ➔ Ajouter import streamlit as st au début du fichier `0_introduction.py`.

Chargement et affichage des données (15 min)

- ➔ Télécharger le jeu de données "Sample Sales Data" à partir de ici.
- ➔ Placer le fichier CSV à la racine du projet.
- ➔ Utiliser pandas pour charger le jeu de données : `df = pd.read_csv("sales_data_sample.csv")`.
- ➔ Afficher les premières lignes du DataFrame à l'aide de `st.dataframe(df.head())`.
- ➔ Ajouter un titre à l'application : st.title("Exploration des données de ventes").

Widgets interactifs (20 min)

- ➔ Ajouter un sélecteur déroulant pour filtrer les données par pays: `country = st.selectbox("Sélectionner un pays", df["COUNTRY"].unique())`.
- ➔ Filtrer le DataFrame en fonction du pays sélectionné : `filtered_df = df[df["COUNTRY"] == country]`.
- ➔ Ajouter un sélecteur déroulant pour filtrer `PRODUCTLINE`
- ➔ Filtrer davantage le DataFrame
- ➔ Afficher le DataFrame filtré : `st.dataframe(filtered_df2)`.

Visualisation de données (adaptées aux colonnes disponibles)

Créer deux colonnes pour avoir un visuel agréable
`col1, col2 = st.columns(2)`

Visualisation de données (20 min)

- ➔ Créer un graphique à barres montrant les ventes totales par produit par client des données filtrées avant, limiter au 10 premier:
- ➔ Utiliser `matplotlib` ou `seaborn` pour créer le graphique.
  - ◆ Afficher le graphique à l'aide de `st.pyplot()`.
- ➔ Créer un graphique linéaire montrant l'évolution des ventes au fil du temps :
  - ◆ Utiliser `plotly` pour créer un graphique interactif.
  - ◆ Afficher le graphique à l'aide de `st.plotly_chart()`.

Mise en page (15 min)

- ➔ Utiliser `st.columns()` pour afficher les graphiques côte à côte.
- ➔ Utiliser `st.expander()` pour afficher les détails de chaque produit:
  `product_details = filtered_df.groupby("PRODUCTLINE")["SALES"].describe()`

Gestion des fichiers (5 min)

- ➔ Ajouter un bouton de téléchargement pour permettre aux utilisateurs de télécharger le DataFrame filtré sous forme de fichier CSV : `st.download_button()`.

Titre du projet : Visualisation interactive d'une fonction affine avec Streamlit

Objectif :

Créer une application Streamlit qui affiche une courbe affine de la forme `y = ax + b`. Les paramètres `a` (pente) et `b` (ordonnée à l'origine) seront contrôlés par des sliders interactifs, et le graphique de la fonction sera mis à jour en temps réel lorsque l'utilisateur modifie les valeurs de `a` ou `b`.

Fonctionnalités :

1. **Interface utilisateur :**
    - Afficher un titre principal avec `st.title()`.
    - Utiliser `st.slider()` pour créer deux sliders, un pour le paramètre `a` et un pour le paramètre `b`.
    - Définir des valeurs minimales, maximales et par défaut appropriées pour les sliders.
2. **Calcul de la fonction :**
    - Générer un ensemble de valeurs `x` sur un intervalle donné (par exemple, de -10 à 10).
    - Calculer les valeurs `y` correspondantes en utilisant la formule `y = ax + b` et les valeurs de `a` et `b` obtenues des sliders.
3. **Affichage du graphique :**
    - Utiliser Matplotlib (`matplotlib.pyplot`) pour créer un graphique de la fonction affine.
    - Afficher le graphique dans l'application Streamlit avec `st.pyplot()`.
4. **Mise à jour en temps réel :**
    - S'assurer que le graphique est mis à jour en temps réel lorsque l'utilisateur modifie les valeurs des sliders.

Instructions :

- Utiliser Streamlit pour créer l'interface utilisateur et gérer les interactions.
- Utiliser NumPy pour générer les valeurs `x` et calculer les valeurs `y`.
- Utiliser Matplotlib pour créer le graphique de la fonction affine.
- Structurer le code de manière claire et concise, en utilisant des commentaires pour expliquer les différentes parties.

Aller plus loin! Au-delà de streamlit!

https://github.com/okld/streamlit-elements

Alternative

https://github.com/Avaiga/taipy

https://github.com/holoviz/panel

https://github.com/posit-dev/py-shiny https://www.gradio.app/